

תיאור התוכנית: file_crypt

סקירה כללית

התוכנית מצפינה ומפענחת קבצים מכל סוג באמצעות אלגוריתם ChaCha20, עם שער כניסה המבוסס על אימות סיסמה. התוכנית כתובה ב-C בלבד, ללא תלות בספריות חיצוניות, ומחולקת ל-5 מודולים: SHA-256, אימות סיסמה, ChaCha20, קלט/פלט קבצים, ו-main.

שיטת ההזדהות

הדרישה קבעה שהסיסמה לא תישמר בקוד באופן גלוי או מוצפן. הפתרון: שמירת **SHA-256 digest** בלבד, כלומר 32 בתים שהם תוצאת פונקציית hash חד-כיוונית על הסיסמה. בזמן ריצה, התוכנית מחשבת SHA-256 על הקלט שהשתמש סיפק ומשווה את התוצאה ל-digest המוטמע. ההשוואה מתבצעת בזמן קבוע (constant-time) כדי למנוע דליפת מידע דרך מדידת זמן התגובה. בתחילה שקלתי להשתמש בהצפנת XOR פשוטה על הסיסמה, אך פסלתי את הגישה כי הצפנה היא הפיכה, ומספיק למצוא את המפתח כדי לשחזר את הסיסמה. SHA-256, לעומת זאת, אינו הפיך אלגוריתמית. שקלתי גם שימוש ב-OpenSSL לצורך חישוב ה-hash, אך העדפתי מימוש עצמאי כדי לא ליצור תלות בספרייה חיצונית ולהדגים הבנה של האלגוריתם.

שיטת ההצפנה

בחרתי ב-**ChaCha20** (לפי RFC 8439), צופן זרם סימטרי שבו ההצפנה והפענוח זהים: XOR של הנתונים עם זרם מפתח פסאודו-אקראי. היתרון המרכזי: אלגוריתם מודרני, פשוט למימוש, ועמיד בפני timing attacks (אין פעולות תלויות-תוכן כמו ב-AES ללא חומרה ייעודית). כל הצפנה מייצרת **nonce אקראי** חדש מ-dev/urandom/ שנכתב כ-header של הקובץ המוצפן, כך ששתי הצפנות של אותו קובץ עם אותו מפתח מניבות ciphertext שונה.

מימושים חלקיים וחולשות ידועות

Key derivation: המפתח ה-ASCII נמתח ל-32 בתים על ידי חזרה מחזורית. זוהי לא פונקציית KDF אמיתית: אין salt ואין work factor. מימוש מלא של PBKDF2-HMAC-SHA256 היה מחזק את ההגנה משמעותית, אך חורג מדרישות המטלה.

אימות שלמות (ChaCha20): MAC לבד מספק סודיות בלבד, ללא הגנה מפני שינוי הציפרטקסט. מימוש מלא היה כולל Poly1305 (כחלק מ-ChaCha20-Poly1305 AEAD), אך גם זה מעבר לנדרש.

בדיקת גודל קובץ: אין הגבלה על גודל קובץ הקלט. קובץ גדול מאוד עלול לגרום לצריכת זמן ודיסק בלתי צפויה.

הגנה מ-TOCTOU: התוכנית לא מוגנת מפני מתקפת TOCTOU (Time-of-Check to Time-of-Use), מצב שבו תוקף מחליף את קובץ הקלט בין הרגע שהתוכנית בודקת אותו לבין הרגע שהיא פותחת אותו לקריאה. מניעה מלאה דורשת נעילת קבצים או פתיחה אטומית ברמת מערכת ההפעלה.

אימות קובץ מוצפן: תוקף בעל גישה למערכת הקבצים יכול להחליף את הקובץ המוצפן בקובץ אחר. ללא חתימה דיגיטלית או MAC על הציפרטקסט, הפענוח יתבצע על קובץ שונה מבלי שהתוכנית תזהה את ההחלפה.

בעיות ופתרונות

Nonce קבוע: הגרסה המקורית השתמשה ב-nonce אפס קבוע, מה שמאפשר שבירה טריוויאלית אם מצפינים שני קבצים באותו מפתח. הפתרון: nonce אקראי חדש לכל הצפנה, מאוחסן בתחילת הקובץ המוצפן.

השוואת סיסמה עם early-exit: הגרסה המקורית השתמשה ב-if על אורך הסיסמה ובהשוואת בתים עם dereference שגוי. הפתרון: השוואת digests (לא הסיסמאות עצמן) באורך קבוע של 32 בתים, ללא הסתעפות תלוית-תוכן.